MLIR NPU Compiler - Lecture 18

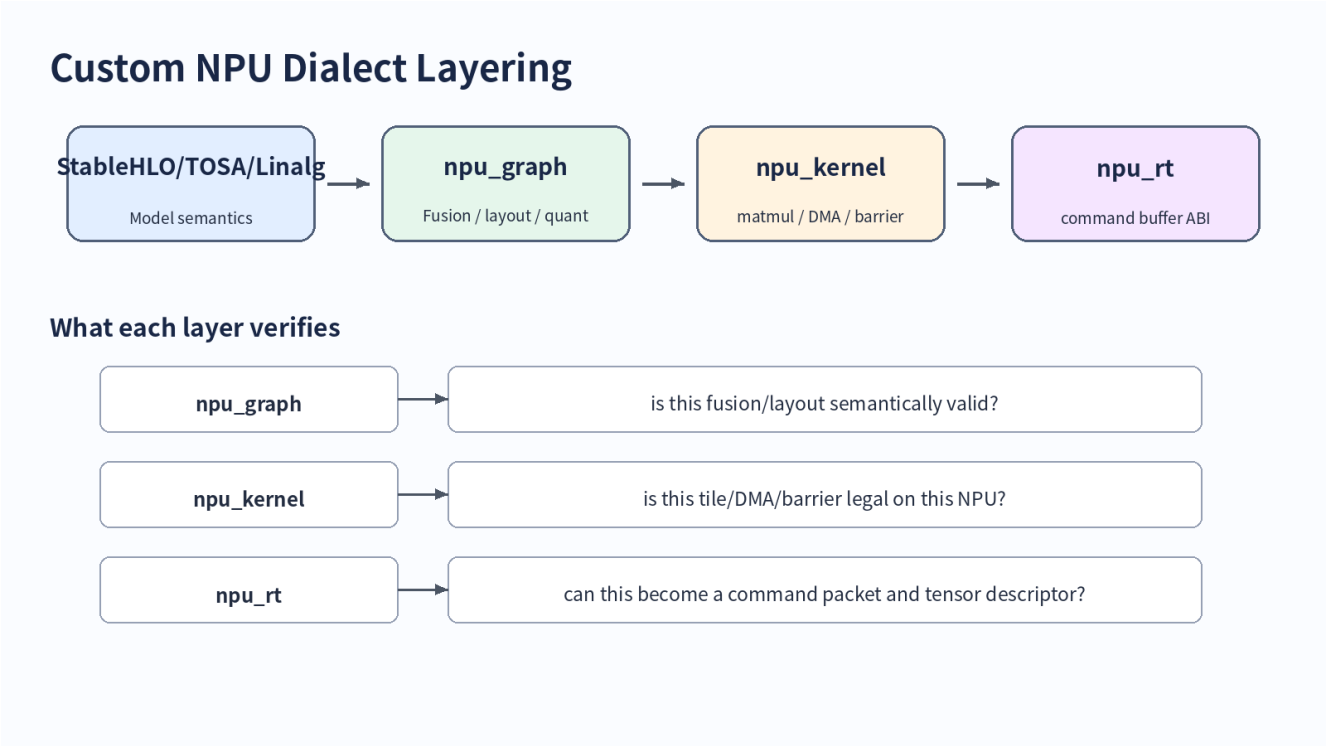
Custom NPU Dialect: npu.matmul / npu.dma / npu.barrier ODS Design

1. 강의 목표

- Custom NPU dialect 가 필요한 위치와 abstraction boundary 를 설명한다.
- ODS/TableGen 으로 operation contract 를 정의하는 방법을 익힌다.
- npu.matmul, npu.dma, npu.barrier 의 operands/results/attributes/verifier 를 설계한다.
- parser/printer, verifier, lowering, FileCheck test 를 하나의 dialect bring-up flow 로 연결한다.

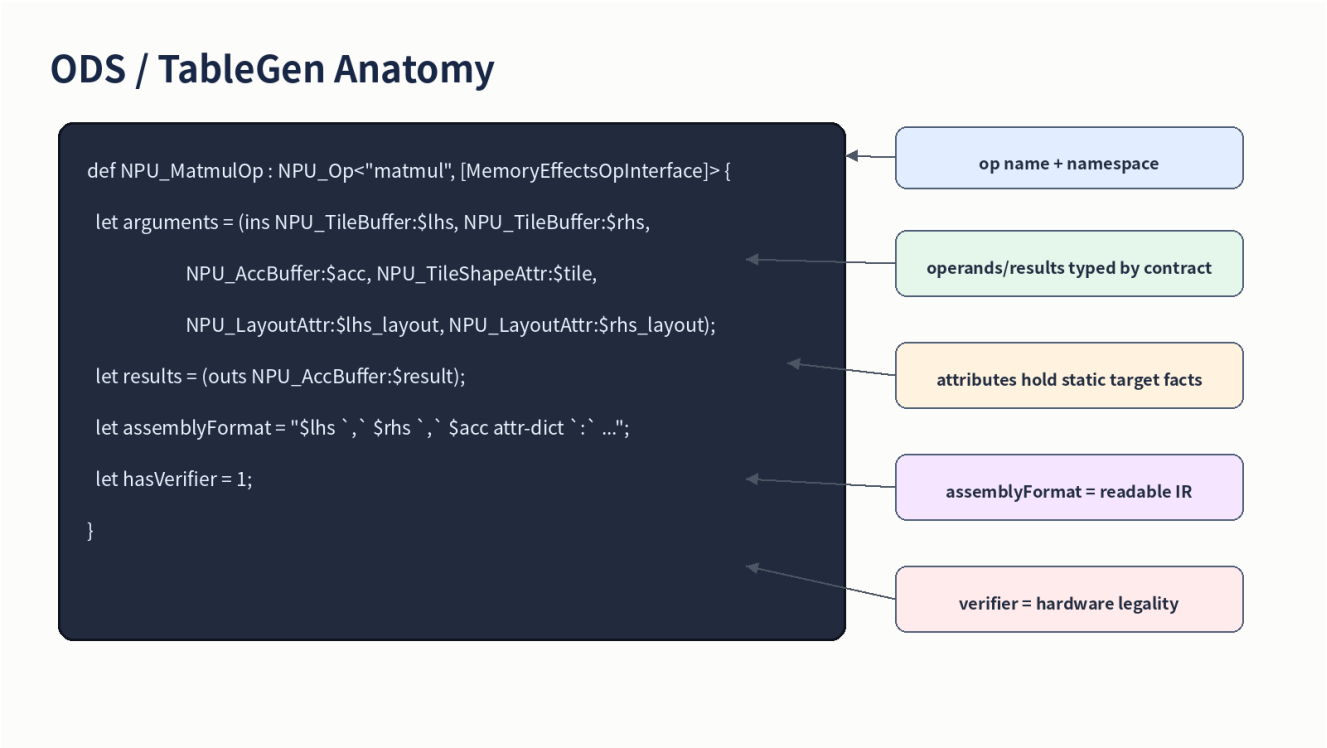
2. 왜 Custom NPU Dialect 인가?

NPU backend 는 native tile shape, SRAM/ACCUM memory space, DMA direction, barrier scope, accumulator width, epilogue policy 와 같은 target contract 를 필요로 합니다. custom dialect 는 이러한 hardware contract 를 operation/type/attribute/verifier 로 고정해 lowering pass 와 test 가 같은 의미를 공유하게 합니다.

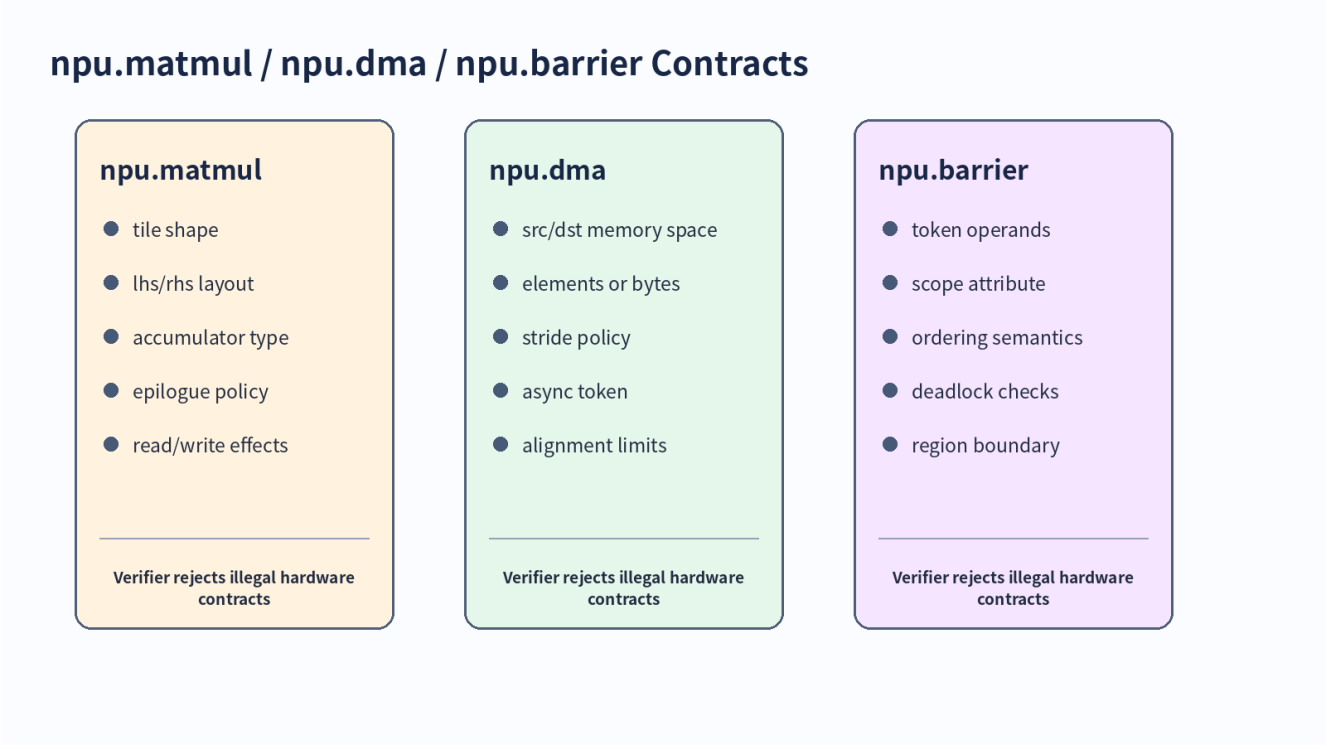


3. ODS/TableGen 핵심

ODS 는 operation 구조의 single source of truth 입니다. op name, documentation, arguments, results, traits/interfaces, assembly format, verifier hook 를 TableGen record 로 선언하고 mlir-tblgen 이 C++ declaration/definition/doc skeleton 을 생성합니다.



4. npu.matmul / npu.dma / npu.barrier contract



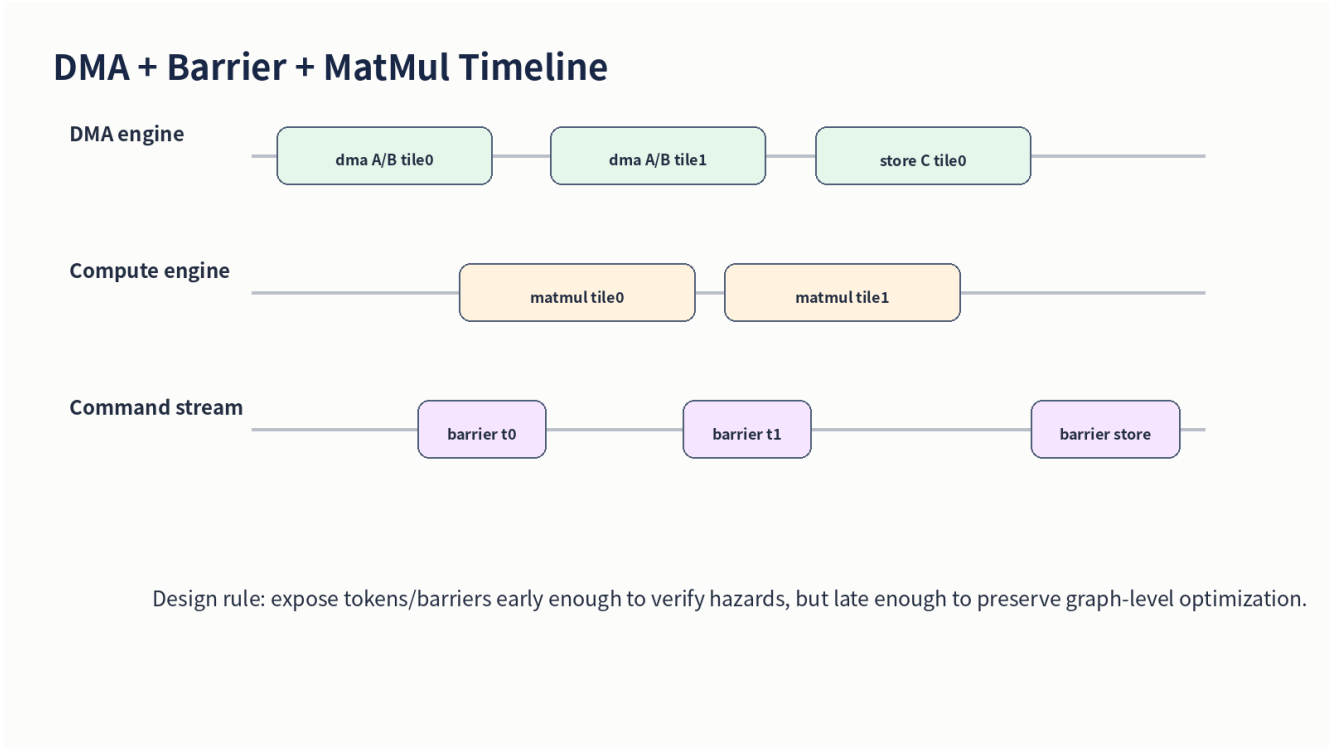
Op	Purpose	Verifier focus
npu.matmul	native tile compute command	tile shape, layout, dtype, accumulator, memory effects
npu.dma	asynchronous memory transfer	src/dst memory space, size, stride, alignment
npu.barrier	dependency/order command	non-empty tokens, scope, dependency consistency

5. ODS skeleton excerpt

```
//===- NPUOps.td - NPU dialect op definitions ----------*- tablegen -*-===//
// Educational skeleton. Adapt constraints and namespaces for a real compiler.

#ifndef NPU_OPS_TD
#define NPU_OPS_TD

include "mlir/IR/OpBase.td"
include "mlir/Interfaces/SideEffectInterfaces.td"
include "mlir/Interfaces/InferTypeOpInterface.td"
include "NPUTypes.td"

def NPU_Dialect : Dialect {
  let name = "npu";
  let cppNamespace = "::mlir::npu";
  let summary = "Target-specific NPU kernel and command dialect";
  let description = [{
    Captures NPU tile-level compute, DMA, and synchronization contracts after
    tiling, layout, memory-space, and quantization decisions have been made.
  }];
}

class NPU_Op<string mnemonic, list<Trait> traits = []> : Op<NPU_Dialect, mnemonic, traits>;

def NPU_MatmulOp : NPU_Op<"matmul", [
    DeclareOpInterfaceMethods<MemoryEffectsOpInterface>,
    DeclareOpInterfaceMethods<InferTypeOpInterface>
  ]> {
  let summary = "Issue one native NPU matrix multiply tile command";
  let arguments = (ins NPU_TileBuffer:$lhs, NPU_TileBuffer:$rhs,
                       NPU_AccBuffer:$acc, NPU_TileShapeAttr:$tile,
                       NPU_LayoutAttr:$lhs_layout, NPU_LayoutAttr:$rhs_layout,
                       OptionalAttr<NPU_EpilogueAttr>:$epilogue);
  let results = (outs NPU_AccBuffer:$result);
  let assemblyFormat = [{
    $lhs `,` $rhs `,` $acc attr-dict `:`
    type($lhs) `,` type($rhs) `,` type($acc) `->` type($result)
  }];
  let hasVerifier = 1;
  let hasCanonicalizer = 1;
}
```

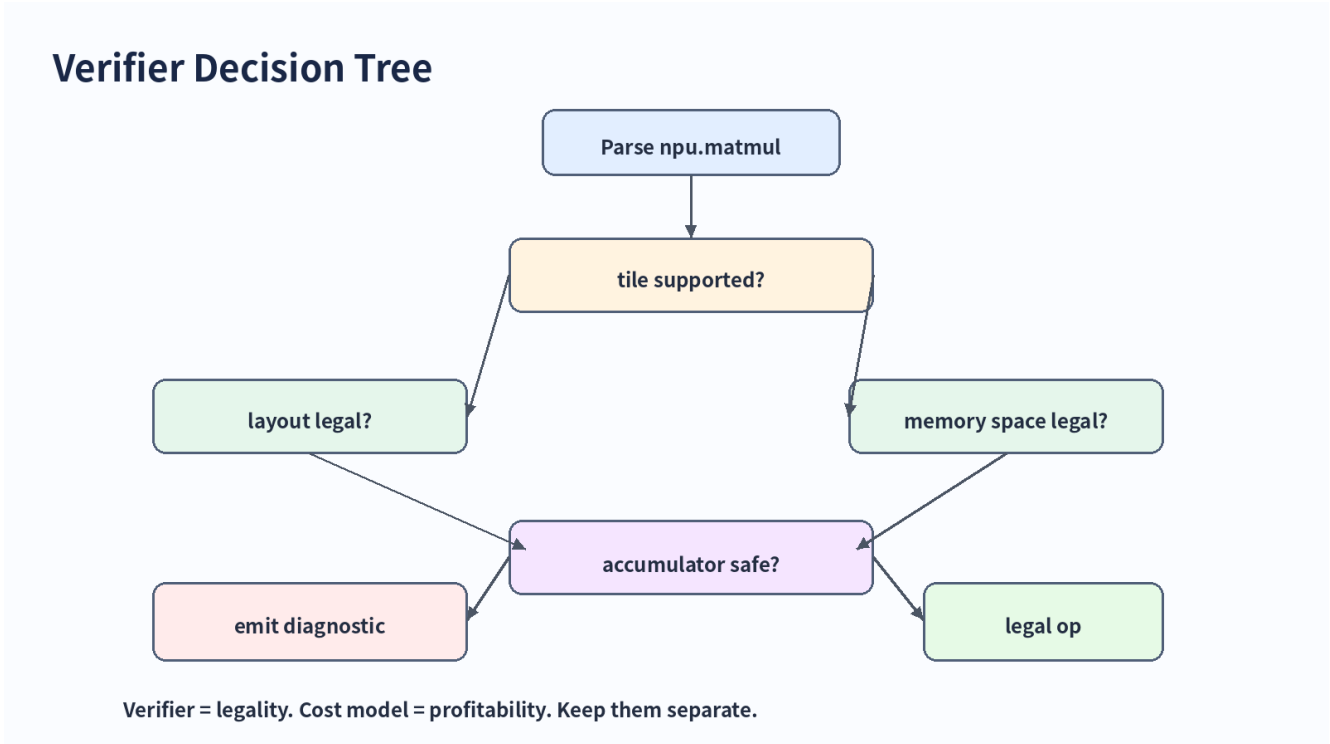
6. DMA + Barrier command scheduling

npu.dma 는 asynchronous transfer 를 시작하고 token 을 반환합니다. npu.barrier 는 token 을 consume 하여 command order 를 명시합니다. 이 모델은 hardware tag-buffer 기반 DMA 로도 낮출 수 있고, command buffer generation 전 hazard verification 에도 유용합니다.



7. Verifier 설계 원칙

Verifier 는 illegal IR 을 reject 해야 합니다. 단순히 느린 schedule 이나 덜 최적인 layout 은 verifier 가 아니라 cost model/fallback policy 가 판단해야 합니다.



8. 실습 과제

1. npu.matmul fields 를 operand/type/attribute 로 분류한다.
2. npu.dma token-based ODS skeleton 을 작성한다.
3. unsupported tile shape negative verifier test 를 작성한다.
4. vector.contract -> npu.matmul lowering legality matrix 를 작성한다.

9. 참고자료

- MLIR Defining Dialects: https://mlir.llvm.org/docs/DefiningDialects/
- MLIR Operation Definition Specification (ODS): https://mlir.llvm.org/docs/DefiningDialects/Operations/
- MLIR Creating a Dialect: https://mlir.llvm.org/docs/Tutorials/CreatingADialect/
- MLIR Customizing Assembly Behavior: https://mlir.llvm.org/docs/DefiningDialects/Assembly/
- MLIR Defining Dialect Attributes and Types: https://mlir.llvm.org/docs/DefiningDialects/AttributesAndTypes/
- MLIR MemRef Dialect: https://mlir.llvm.org/docs/Dialects/MemRef/
- MLIR Testing Guide: https://mlir.llvm.org/getting_started/TestingGuide/